FPT UNIVERSITY

Graduate School of Information Technology

Master of Software Engineering — Artificial Intelligence

HarnessEval: Evaluating Coding Agent Scaffolds Through Ablation-Driven Multi-Dimensional Analysis

Research Proposal

Course: Methods of Learning and Scientific Research

Student: [Student Name]
Student ID: [Student ID]
Supervisor: [Supervisor Name]
Date: April 2026

Ho Chi Minh City, 2026

Contents

1 Introduction **3**

1.1 Problem Statement 3

1.2 Research Purpose (Qualitative) 4

1.3 Research Objectives (Quantitative) 4

1.4 Research Hypotheses 5

1.5 Scope and Delimitations 5

1.6 Expected Thesis Structure 5

2 Literature Review and Theoretical Background **6**

2.1 Literature Review 6

2.1.1 Coding Agent Harness Architecture 6

2.1.2 Coding Agent Benchmarks 6

2.1.3 Tool Learning and Context Management 7

2.1.4 Summary on Characteristics Investigated 7

2.1.5 Research Gap Summary 9

2.2 Theoretical Background 9

2.2.1 Harness Architecture: Five-Layer Model 9

2.2.2 Ablation Studies in Software Engineering 10

2.2.3 Defining “Independent of LLM” via Three-Way ANOVA 10

3 Research Methodology **11**

3.1 Overview 11

3.2 Three-Dimensional Harness Evaluation Taxonomy (RO1) 11

3.3 Modular Harness via SWE-Agent Fork (RO2) 12

3.4 Pilot Study (RO3, Phase 1) 12

3.5 Full Ablation Study (RO3, RO4, Phase 2) 13

3.6 Threats to Validity 13

3.6.1 Internal Validity 13

3.6.2 External Validity 14
3.6.3 Construct Validity 14
3.7 Risk Analysis 15

4 Expected Results **15**
4.1 Expected Outcomes per Objective 15
4.2 Scientific Significance 16
4.3 Practical Significance 16
4.4 Publication Targets 16

5 Timeline and Budget **17**
5.1 Timeline 17
5.2 Budget 17

References **18**

1. Introduction

1.1. Problem Statement

Large Language Model (LLM)-based coding agents such as Claude Code, GitHub Copilot, Cursor, and SWE-Agent have become integral tools in modern software development. These agents autonomously read source code, fix bugs, generate features, and interact with file systems, terminals, and version control systems [1, 16].

However, the performance of a coding agent depends on two distinct factors: (1) the **underlying LLM** (e.g., Claude, GPT, DeepSeek) and (2) the **harness**—the infrastructure scaffold comprising the tool dispatch system, context management layer, and supporting components [1]. Lou et al. [2] demonstrated that the harness plays a critical role: a smaller model with a well-designed harness can outperform a larger model without one, eliminating 100% of illegal actions in constrained environments. Bui et al. [1] described the harness as an "operating system for the agent," featuring a multi-layer architecture including dual-agent design, Adaptive Context Compaction (ACC) that reduces token usage by 54%, and event-driven system reminders to combat instruction fade-out.

The current paradox. The research community has developed numerous benchmarks for evaluating agent *output*—SWE-Bench Pro with 1,865 tasks [5], SWE-Compass with 2,000 instances across 10 languages [9], and PRDBench with agent-driven evaluation [8]. However, an analysis of 49 recent papers (2023–2026) reveals that **no study systematically evaluates the harness itself** [19]. When reporting "Agent A achieves 70% resolve rate," we do not know which harness component contributed to that result.

Why this is a genuine research gap. A legitimate question is: why have the groups that develop harnesses (Princeton—SWE-Agent, Anthropic—Claude Code) not conducted this evaluation themselves? Three reasons explain this gap:

1. **Different roles.** They are *developers* with incentives to build new harnesses, not *evaluators* with incentives to compare systems—analogous to how automobile manufacturers build cars while independent organizations (Euro NCAP, IIHS) conduct safety evaluations.

2. **Rapid field growth.** From 2023 to 2026, the number of coding agents grew from a handful to dozens. No group has paused to meta-evaluate because the field is in a construction race.

3. **Indirect evidence of need.** Lou et al. [2] compared harnesses on game tasks; Robeyns et al. [4] self-improved a harness without measuring individual component contributions—demonstrating that the need exists but has not been formalized.

Research gap. There is no evaluation framework that enables: (a) measuring the quality of

individual harness components, (b) decomposing the contributions of harness vs. LLM, and (c) comparing different harnesses across unified quality dimensions.

1.2. Research Purpose (Qualitative)

This study aims to **construct the first integrated evaluation framework** for coding agent harness infrastructure, enabling the decomposition and quantification of individual harness component contributions to overall agent performance, independently of the underlying LLM capability.

1.3. Research Objectives (Quantitative)

RO1.

Design a **3-dimensional harness evaluation taxonomy** (Tool Dispatch Efficiency, Context Utilization, Backend Portability) with 7 metrics total, validated by at least 5 domain experts (inter-rater agreement measured by Fleiss' $\kappa \geq 0.6$).

RO2.

Fork and refactor SWE-Agent into a **modular harness** with swappable components (tool set, context strategy, model backend), ensuring resolve rate **equivalent to baseline SWE-Agent** ($\pm 3\%$ absolute) on 50 verification tasks.

RO3.

Conduct a **systematic ablation study** on 150 tasks from SWE-Bench Verified with 27 conditions ($3 \times 3 \times 3$ full factorial: tool configurations $\times$ context strategies $\times$ LLM backends), including a pilot study (5 conditions $\times$ 20 tasks) prior to the full experiment. Report mean $\pm$ standard deviation from 3 runs for the 10 most critical conditions.

RO4.

Decompose variance contributions of harness vs. LLM using **three-way ANOVA** (Tool $\times$ Context $\times$ Backend), with statistical significance at $p < 0.05$ after Bonferroni correction. Include a **Generalized Linear Mixed Model** (GLMM) as a robustness check for binary outcomes.

RO5.

Release an **open-source evaluation toolkit** on GitHub with documentation, examples, and CI/CD pipeline.

1.4. Research Hypotheses

H1. The tool system has the **largest effect size** (Cohen's $d > 0.5$) among the three harness components on resolve rate.

H2. Context management has a **medium effect size** (Cohen's $d = 0.3$–0.5).

H3. Harness configuration explains at least **10% of resolve rate variance** ($\eta^2 \geq 0.10$ in ANOVA), independent of LLM backend.

H4. Resolve rate variance **across LLM backends decreases** when harness quality is higher (statistically significant interaction effect).

1.5. Scope and Delimitations

In scope:

- Harness/scaffold infrastructure of LLM-based coding agents
- Three harness components: tool system, context manager, model backend
- 150 tasks from SWE-Bench Verified (Python)
- Three LLM backends: Claude Sonnet 4, GPT-4o, DeepSeek-V3
- Base harness: SWE-Agent (forked and refactored)

Out of scope (future work):

- Safety/security evaluation → separate study (SecureCodeBench)
- Session continuity / long-horizon evaluation
- Languages other than Python
- Commercial harnesses (Claude Code, Cursor)

1.6. Expected Thesis Structure

1. **Chapter 1:** Introduction (problem, literature review, objectives, scope)
2. **Chapter 2:** Theoretical Background (agent architecture, ablation methodology, ANOVA)
3. **Chapter 3:** Research Methodology (taxonomy design, modular harness, ablation protocol)

4. **Chapter 4:** Results and Discussion
5. **Chapter 5:** Conclusion and Future Work

2. Literature Review and Theoretical Background

2.1. Literature Review

2.1.1. Coding Agent Harness Architecture

Wang et al. [18] proposed the Profile-Memory-Planning-Action framework for LLM agents, cited over 3,000 times. Bui et al. [1] instantiated this framework for coding agents with OpenDev—a terminal-native harness featuring dual-agent architecture (Planner-Executor), 5-stage Adaptive Context Compaction reducing token usage by 54%, and event-driven system reminders. Lou et al. [2] demonstrated that harnesses can be automatically synthesized, enabling smaller models to outperform larger ones. Cao et al. [3] showed that coding agents process long contexts effectively by externalizing reasoning through file-system tools, outperforming SOTA by 17.3%. Robeyns et al. [4] proposed SICA—a self-improving coding agent that increased SWE-Bench from 17% to 53% over 15 iterations—but did not measure individual component contributions.

Observation: These works describe harness designs in detail but **do not benchmark** individual components.

2.1.2. Coding Agent Benchmarks

SWE-Bench Pro [5] provides 1,865 contamination-resistant tasks from commercial codebases. SWE-EVO [6] evaluates long-horizon software evolution (best model achieves only 21%). SWE-Compass [9] covers 10 programming languages and 8 task types. Prathifkumar et al. [7] found evidence that LLMs may memorize patches rather than reason—Claude performs 3–6$\times$ better on file localization in SWE-Bench vs. novel benchmarks. AgentBoard [15] introduced the progress rate metric with Pearson correlation > 0.95 with human judgment. A survey on agent evaluation [19] found that 8 out of 10 popular benchmarks have serious validity issues—do-nothing agents pass 38% of tasks.

Observation: All benchmarks evaluate **output quality**. None evaluates **harness quality**.

2.1.3. Tool Learning and Context Management

ToolLLM [12] evaluates 16,000+ APIs with DFSDT reasoning. AnyTool [13] proposes hierarchical self-reflective API retrieval. EasyTool [14] reduces token cost by 70%+ through documentation simplification. Mem0 [10] achieves 91% latency reduction for persistent memory. A-MEM [11] uses a Zettelkasten-inspired approach. Wang et al. [17] discovered that agents repeat the same bugs across iterations due to lacking persistent memory.

Observation: Each paper evaluates **a single component in isolation**. No study evaluates **all components simultaneously** within a harness and compares their relative contributions.

2.1.4. Summary on Characteristics Investigated

Table 1 presents a systematic comparison of the characteristics investigated by related works, following the format of Ho-Van (2013). Each column represents a key aspect of harness evaluation. The table reveals that while individual aspects have been partially addressed in isolation, **no prior work addresses all characteristics simultaneously**. In particular, columns C3 (Backend Portability), C4 (Component Ablation), and C5 (Harness vs. LLM Decomposition) are **entirely empty**—no paper has investigated these aspects.

Table 1: Summary on Characteristics Investigated by Related Works

Ref	Paper	C1 Tool Dispatch	C2 Context Eval	C3 Backend Port.	C4 Ablation	C5 H vs L Decomp.	C6 Multi-Metric	C7 Coding-Specific	C8 Open-Source
[1]	Bui 2026 (OpenDev)	△	△					✓	✓
[2]	Lou 2026 (AutoHarness)	△			△				
[3]	Cao 2026 (LongContext)	✓	△					✓	
[4]	Robeyns 2025 (SICA)	△						✓	✓
[5]	Deng 2025 (SWE-Bench Pro)							✓	△
[9]	Xu 2025 (SWE-Compass)						△	✓	✓
[10]	Chhikara 2025 (Mem0)		✓*						✓
[11]	Xu 2025 (A-MEM)		✓*						✓
[?]	Yu 2026 (AgeMem)		✓*		△				
[?]	Patil 2023 (Gorilla)	✓†							✓
[12]	Qin 2023 (ToolLLM)	✓†							✓
[13]	Du 2024 (AnyTool)	✓†							
[15]	Ma 2024 (AgentBoard)	△	△				✓		✓
[17]	Wang 2025 (VisualAnalytics)		△				✓	✓	
[19]	2025 (EvalSurvey)						△		
	Ours (HarnessEval)	✓	✓	✓	✓	✓	✓	✓	✓

✓ = Addressed △ = Partially addressed (blank) = Not addressed

* Context evaluated on chatbot benchmarks (LoCoMo), not coding tasks

† Tool accuracy evaluated in isolation (API benchmarks), not within a coding harness

Key observations from Table 1:

1. **Three columns entirely empty (C3, C4, C5):** No prior work evaluates backend portability, conducts component ablation on coding harnesses, or decomposes harness vs. LLM contributions. These represent the strongest evidence for our research gap.

2. **Tool evaluation exists but outside harness context (C1):** Gorilla [?], ToolLLM [12], and AnyTool [13] evaluate tool accuracy on standalone API benchmarks (✓†), not within an integrated coding harness where tool dispatch interacts with context management and orchestration.

3. **Context evaluation exists but wrong domain (C2):** Mem0 [10], A-MEM [11], and AgeMem [?] evaluate memory/context on chatbot benchmarks (✓*) such as LoCoMo and DialSim—not on coding tasks where context includes AST structures, dependency graphs, and test results.

4. **Multi-metric frameworks exist but evaluate output, not harness (C6):** AgentBoard [15] provides multi-dimensional evaluation via progress rates, but measures *agent output quality*, not *harness infrastructure quality*.

5. **HarnessEval is the first to address all 8 characteristics simultaneously,** filling the gap between isolated component evaluations and output-only benchmarks.

2.1.5. Research Gap Summary

Based on the characteristics analysis above, Table 2 summarizes the research gap.

Table 2: Research Gap Summary

Aspect	Existing Work	Gap (This Study)
Agent output evaluation	SWE-Bench, SWE-Compass, AgentBoard	—
Tool accuracy (isolated)	ToolLLM, AnyTool, Gorilla	Tool dispatch **within** coding harness
Context/memory (chatbot)	Mem0, A-MEM (LoCoMo benchmark)	Context evaluation **on coding tasks**
Harness description	OpenDev, AutoHarness	**Measuring** harness quality with metrics
Cross-backend comparison	—	**Entirely unexplored**
Component ablation (coding)	—	**Entirely unexplored**
Harness vs. LLM decomposition	—	**Entirely unexplored**

2.2. Theoretical Background

2.2.1. Harness Architecture: Five-Layer Model

Based on [18] and [1], a coding agent harness comprises five layers:

1. **Layer 1 — Model Backend:** The underlying LLM (Claude, GPT, DeepSeek)
2. **Layer 2 — Tool System:** Available tools (Read, Write, Edit, Bash, Grep, Glob, etc.) and dispatch logic
3. **Layer 3 — Context & Memory Management:** Context window management, compaction, persistent memory
4. **Layer 4 — Safety & Permission:** Sandboxing, permission model, input validation (future work)
5. **Layer 5 — Orchestration:** Agent loop (ReAct, Plan-Execute), multi-turn management (future work)

This study evaluates **Layers 1–3**. Layers 4–5 are designated as future work.

2.2.2. Ablation Studies in Software Engineering

Ablation study is a standard methodology in machine learning: systematically removing or modifying components to measure their individual contributions. Applied to harness evaluation:

- Modify tool set (12 → 8 → 5 tools) → measure resolve rate change
- Change context strategy (full → sliding window → summary) → measure resolve rate change
- Switch LLM backend (Claude → GPT → DeepSeek) → measure portability

Confounding variables. When removing a tool, performance may decrease because (a) the agent lacks the capability, or (b) the harness dispatches poorly. To address this: we *reduce* tools gradually rather than removing entirely, and record which tasks *require* the removed tool to distinguish the two causes.

2.2.3. Defining "Independent of LLM" via Three-Way ANOVA

"Independent of LLM" does **not** mean the harness and LLM do not interact. It means their contributions can be **statistically decomposed**.

Method: Three-way ANOVA (following reviewer feedback to use 3-way rather than 2-way):

- Factor A: Tool Configuration (3 levels)
- Factor B: Context Strategy (3 levels)
- Factor C: LLM Backend (3 levels)
- Dependent variable: Resolve rate

$$\text{Total Variance} = \underbrace{V_{\text{Tool}} + V_{\text{Context}}}_{\text{Harness}} + V_{\text{Backend}} + \underbrace{V_{\text{T}\times\text{B}} + V_{\text{C}\times\text{B}}}_{\text{Interactions}} + V_{\text{T}\times\text{C}} + V_{\text{T}\times\text{C}\times\text{B}} + V_{\text{Error}} \tag{1}$$

If V_{Tool} and V_{Context} are statistically significant, the harness has an independent effect. The effect size η^2 indicates the percentage of variance explained by each factor. This formalization is the **primary theoretical contribution** of this study.

Robustness check: Since resolve rate per task is binary (0/1), ANOVA assumptions may be violated. A Generalized Linear Mixed Model (GLMM) with binary outcome and random effects for tasks will be used as a robustness check. Both ANOVA (for interpretability) and GLMM (for statistical correctness) will be reported.

3. Research Methodology

3.1. Overview

This study employs an **experimental methodology** with four phases:

1. Taxonomy design and expert validation
2. SWE-Agent fork and modular refactoring
3. Pilot study (5 conditions $\times$ 20 tasks)
4. Full ablation study (27 conditions $\times$ 150 tasks)

3.2. Three-Dimensional Harness Evaluation Taxonomy (RO1)

Table 3: HarnessEval Taxonomy: 3 Dimensions, 7 Metrics

Dim.	ID	Metric	Formal Definition	Auto?
D1. Tool Dispatch	M1.1	Correct Selection	% calls selecting an acceptable tool (from set of valid options, not single ground-truth)	No
	M1.2	Redundant Call Rate	% calls whose output is unused within 3 subsequent turns	Yes
	M1.3	Utilization Breadth	Unique tool types used / total tools available	Yes
D2. Context Utilization	M2.1	Info Retention	BERTScore between full-context and compacted-context responses on same task	Yes
	M2.2	Effective Token Ratio	% tokens classified as task-relevant by LLM classifier (validated on 200 segments with human labels, acceptance threshold: accuracy $> 90\%$)	Semi
D3. Backend Portability	M3.1	Cross-Backend StdDev	σ(resolve rates across 3 backends)	Yes
	M3.2	Min/Max Ratio	$\min(\text{RR})/\max(\text{RR})$ across backends	Yes

Validation procedure: The taxonomy will be presented to 5 domain experts (at least 2 from industry, at least 2 from academia). Each expert independently rates 20 sample evaluations. Inter-rater agreement will be measured using Fleiss' κ, with a target of $\kappa \geq 0.6$.

3.3. Modular Harness via SWE-Agent Fork (RO2)

SWE-Agent is an open-source (MIT license) coding agent with approximately 15,000 LOC. We fork and refactor three modules:

- `tool_config`: Swappable tool sets (12/8/5 tools) via configuration file
- `context_strategy`: Swappable strategies (full/sliding-window/summary) via configuration file
- `model_backend`: Swappable LLM backends (Claude/GPT/DeepSeek) via configuration file

Acceptance criteria: The modular harness must achieve resolve rate within $\pm 3\%$ absolute of baseline SWE-Agent on 50 verification tasks. The orchestration logic remains unchanged to ensure baseline quality.

Estimated effort: 2–3 weeks (vs. 1–3 months if built from scratch).

3.4. Pilot Study (RO3, Phase 1)

Prior to committing to the full experiment:

Table 4: Pilot Study Design

Parameter	Value
Conditions	5 (Full, Reduced tools, No context mgmt, GPT backend, DeepSeek backend)
Tasks	20 (randomly sampled from 150)
Runs per condition	2 (for reproducibility estimate)
Total evaluations	$5 \times 20 \times 2 = 200$
Estimated cost	\$50–80
Duration	1–2 weeks

Pilot objectives:

1. Validate that metrics discriminate between conditions
2. Estimate accurate cost for the full experiment
3. Debug the evaluation pipeline (Docker, API, logging)
4. Compute preliminary effect sizes to confirm power analysis

3.5. Full Ablation Study (RO3, RO4, Phase 2)

Table 5: Full Factorial Experimental Design

Factor	Levels	Details
Tool Configuration	3	Full (12 tools) / Medium (8) / Minimal (5)
Context Strategy	3	Full context / Sliding window (50K tokens) / Summary-based
LLM Backend	3	Claude Sonnet 4 / GPT-4o / DeepSeek-V3
Total conditions	**27**	$3 \times 3 \times 3$ full factorial

Dataset: 150 tasks from SWE-Bench Verified, selected via stratified random sampling by difficulty (easy/medium/hard based on number of models that solved each task). Tasks with unstable Docker environments are excluded via pre-filtering.

Runs:

- 10 most important conditions: 3 runs each (report mean $\pm$ std)
- 17 remaining conditions: 1 run each
- Total: $(10 \times 3 + 17 \times 1) \times 150 = 7{,}050$ evaluations

Statistical analysis (RO4):

- **Three-way ANOVA:** Tool (3) $\times$ Context (3) $\times$ Backend (3)
- Report: F-statistic, p-value (Bonferroni-corrected), η^2 (eta-squared)
- Post-hoc: Tukey HSD for pairwise comparisons
- Effect size: Cohen's d for each pairwise comparison
- **Robustness check:** GLMM with binary outcome (solved/not solved), random effects for tasks, fixed effects for Tool, Context, and Backend
- Power analysis: with 150 tasks, 27 conditions, $\alpha = 0.05$, power ≥ 0.80

3.6. Threats to Validity

3.6.1. Internal Validity

- **Confounding:** Reducing tools may change the problem, not just harness quality. *Mitigation:* Reduce tools gradually (12→8→5); record which tasks require removed tools; exclude those from component analysis.

- **LLM non-determinism:** Stochastic outputs introduce variance. *Mitigation:* Set temperature $= 0$; run 3 times for critical conditions; report confidence intervals.
- **Docker instability:** Some SWE-Bench environments are unstable. *Mitigation:* Pre-filter tasks; retry twice on failure.
- **LLM versioning:** Models may be updated during the study. *Mitigation:* Record exact model version/snapshot; run all experiments within a short time window.

3.6.2. External Validity

- **Python only:** Tool systems for Python (pytest, pip) differ from Java (Maven, JUnit) or Rust (Cargo). Results require caution when generalizing.
- **Open-source only:** Commercial harnesses (Claude Code, Cursor) may exhibit different behavior.
- **Survivor bias:** Excluded tasks (Docker failures) may be harder tasks where harness matters more.

3.6.3. Construct Validity

- **M1.1** depends on annotator judgment. *Mitigation:* 2 annotators, Fleiss' κ.
- **M2.2** uses an LLM classifier. *Mitigation:* Validate on 200 segments with human labels; acceptance threshold: accuracy $> 90\%$.
- **Statistical conclusion validity:** Multiple testing across 7 metrics requires Bonferroni or Benjamini-Hochberg FDR correction.

3.7. Risk Analysis

Table 6: Risk Assessment and Mitigation

Risk	Prob.	Impact	Mitigation
Docker failures on many tasks	Medium	Reduced sample	Pre-filter; backup tasks
API costs exceed budget	High	Cannot complete	Pilot estimates first; use DeepSeek (cheapest)
Modular harness >3% below SWE-Agent	Low	RO2 failure	Minimal refactoring; keep original code
Metrics do not discriminate	Medium	Taxonomy invalid	Pilot detects early; revise metrics
Experts unavailable for validation	Medium	RO1 delayed	Online survey; increase expert pool
DeepSeek API instability	Low	Lose 1 backend	Backup: Qwen-2.5-72B or Llama-3.1-70B

4. Expected Results

4.1. Expected Outcomes per Objective

RO1 — Taxonomy: A validated 3-dimensional, 7-metric taxonomy with Fleiss' $\kappa \geq 0.6$.

RO2 — Modular Harness: A fork of SWE-Agent with modular components, resolve rate within $\pm 3\%$ of baseline.

RO3 — Ablation Study: Empirical results answering:

- Which component contributes most to resolve rate? (**H1**: Tool system has largest effect)
- How does context management affect performance? (**H2**: Medium effect size)
- What percentage of variance does harness explain? (**H3**: $\eta^2 \geq 0.10$)
- Does a better harness reduce cross-backend variance? (**H4**: Significant interaction)

Whether hypotheses are supported or rejected, both outcomes constitute scientific contributions.

RO4 — ANOVA Decomposition: Variance decomposition table showing harness vs. LLM contributions.

RO5 — Toolkit: Open-source Python package on GitHub with documentation and CI/CD.

4.2. Scientific Significance

- **Theoretical:** Formalization of harness quality decomposition via ANOVA (first in this domain)
- **Empirical:** First quantification of individual harness component contributions
- **Methodological:** Reusable evaluation framework and toolkit

4.3. Practical Significance

- **Developers:** Know which harness component to prioritize for improvement
- **Researchers:** Standardized framework for comparing new harnesses
- **Industry:** Evidence-based guidelines for harness design

4.4. Publication Targets

Table 7: Target Publication Venues

Venue	Type	Fit
ICSE 2027 NIER	Conference (New Ideas)	Strong — coding agent + evaluation
MSR 2027	Conference	Good — mining SE data
LLM Agents Workshop	Workshop (NeurIPS/ICML)	Good — agent evaluation
EMNLP Demo Track	Conference	Good — toolkit demonstration

5. Timeline and Budget

5.1. Timeline

Table 8: Research Timeline (24 Weeks)

Phase	Weeks	Activities	Deliverable
GD1	1–6	Taxonomy design + expert validation (5 experts, Fleiss' κ)	Taxonomy v1.0
GD2	7–9	Fork SWE-Agent + modular refactoring + verify $\pm 3\%$	Modular harness v1.0
Pilot	10–11	Pilot study (200 evaluations)	Pilot report
GD3	12–18	Full ablation (7,050 evaluations)	Raw data, ANOVA
GD4	19–22	Analysis + guidelines + thesis writing	Thesis draft
GD5	23–24	Review, revision, toolkit release	Final thesis, GitHub

5.2. Budget

Table 9: Budget Estimate

Item	Calculation	Cost (USD)
Pilot study	200 evals $\times$ \$0.27 + retries	\$80
Full experiment	7,050 evals $\times$ \$0.27 + 30% retries	\$2,500
Contingency (20%)		\$500
Total		**\$2,500–3,100**

References

References

[1] N. D. Q. Bui et al., "Building Effective AI Coding Agents for the Terminal: Scaffolding, Harness, Context Engineering, and Lessons Learned," *arXiv:2603.05344*, 2026.

[2] X. Lou et al., "AutoHarness: Improving LLM Agents by Automatically Synthesizing a Code Harness," *arXiv:2603.03329*, 2026.

[3] W. Cao et al., "Coding Agents are Effective Long-Context Processors," *arXiv:2603.20432*, 2026.

[4] M. Robeyns et al., "A Self-Improving Coding Agent," *arXiv:2504.15228*, 2025.

[5] X. Deng et al., "SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks?" *arXiv:2509.16941*, 2025.

[6] M. V. T. Thai et al., "SWE-EVO: Benchmarking Coding Agents in Long-Horizon Software Evolution Scenarios," *arXiv:2512.18470*, 2025.

[7] T. Prathifkumar et al., "Does SWE-Bench-Verified Test Agent Ability or Model Memory?" *arXiv:2512.10218*, 2025.

[8] L. Fu et al., "Automatically Benchmarking LLM Code Agents through Agent-Driven Annotation and Evaluation," *arXiv:2510.24358*, 2025.

[9] J. Xu et al., "SWE-Compass: Towards Unified Evaluation of Agentic Coding Abilities for Large Language Models," *arXiv:2511.05459*, 2025.

[10] P. Chhikara et al., "Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory," *arXiv:2504.19413*, 2025.

[11] W. Xu et al., "A-MEM: Agentic Memory for LLM Agents," *arXiv:2502.12110*, 2025.

[12] Y. Qin et al., "ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs," *arXiv:2307.16789*, ICLR 2024.

[13] Y. Du et al., "AnyTool: Self-Reflective, Hierarchical Agents for Large-Scale API Use," *arXiv:2402.04253*, ACL 2024.

[14] S. Yuan et al., "EASYTOOL: Enhancing LLM-based Agents with Concise Tool Instruction," *arXiv:2401.06201*, ACL Findings 2024.

[15] C. Ma et al., "AgentBoard: An Analytical Evaluation Board of Multi-Turn LLM Agents," *arXiv:2401.13178*, NeurIPS 2024.

[16] "AI Agentic Programming Survey," *arXiv:2508.11126*, 2025.

[17] J. Wang et al., "Illuminating LLM Coding Agents: Visual Analytics for Deeper Understanding and Enhancement," *arXiv:2508.12555*, IEEE TVCG 2025.

[18] L. Wang et al., "A Survey on Large Language Model based Autonomous Agents," *arXiv:2308.11432*, Frontiers of Computer Science 2025.

[19] "Survey on Evaluation of LLM-based Agents," *arXiv:2503.16416*, 2025.